Case Study: Convert Iterative to Recursive Implementation

Problem Statement: Students are asked to write a program that uses loops to print the multiplication table (sifir) of a given number. For example, the function `Sifir(2, 12)` should produce the multiplication table for 2 up to 12 (i.e., 2 × 1 = 2, 2 × 2 = 4, ..., 2 × 12 = 24).

The task for students is to implement an **iterative approach (using a loop)**, and then convert it into a **recursive implementation**.

Part 1: Iterative Implementation (Using `while` loop)

Below is a C++ implementation of the multiplication table (`Sifir`) using a `while` loop:

```
cpp
Copy code
#include <iostream>
using namespace std;

void Sifir(int num, int limit) {
    int i = 1;  // Start from 1
    while (i <= limit) {
        cout << num << " x " << i << " = " << num * i << endl;
        i++;
    }
}

int main() {
    int num = 2;
    int limit = 12;
    Sifir(num, limit);  // Prints the sifir of 2 from 1 to 12
    return 0;
}
```

Output for `Sifir(2, 12)`:

```
Copy code
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18
2 x 10 = 20
2 x 11 = 22
2 x 12 = 24
```

Part 2: Conversion to Recursive Implementation

Now, students are asked to convert the above code into a **recursive implementation**. Instead of using a loop to increment and print each multiplication, we will use recursive function calls.

Recursive Implementation:

In recursion, we need a **base case** and a **recursive step**. The base case occurs when the counter (`i`) exceeds the limit, and the recursive step is when we print the current multiplication and call the function with `i + 1`.

Please try to solve this case study using RECURSIVE.

Student answer (code) :

```
#include <iostream>

using namespace std;



int Sifir (int num, int limit, int i){

   if (i>limit)

      return 0;

   else {

      cout << num << "x" << i << "=" << num*i << endl;

      return Sifir (num, limit, i+1);

}

}



int main (){

   int num = 2;

   int limit = 12;

   int x = Sifir (num, limit, 1);
```

```
    return 0;

}
```

Conclusion:

- **Iterative Implementation:** The `while` loop iterates from `i = 1` to `limit`, printing the multiplication table.
- **Recursive Implementation:** The recursive function prints one multiplication at a time and calls itself to handle the next value of `i`.

The key challenge for students is understanding how to replace iteration with recursion by breaking the problem down into smaller sub-problems and ensuring they have both the base case and recursive step properly defined.


UTM
UNIVERSITI TEKNOLOGI MALAYSIA